Live Imaging to Point Cloud Data

Cole Miller & Eric Son

G.W. Woodruff School of Mechanical Engineering

Georgia Institute of Technology, Atlanta, GA 30332

I. Introduction/Background

CAD software such as SOLIDWORKS, NX, and Creo are some of the many tools to generate 3D models for engineering. However, there are not many instances where the model can be immediately generated from our surroundings. Using a computer webcam to identify varying objects and geometries and achieve a sense of depth perception are the main objectives of this project. The main goal is to ultimately identify any available object and import the data for the computer to recognize it as a 3D model. This is important because this project will take a 2D image from MATLAB (or Python) and allow a 3D model to be generated in Python using meshes, which is not a common design approach seen in the engineering environment. The minimum requirement of depth perception of any object shown in a 2D figure is to be expected by the end of this analysis. From there, there are hopes of generating 3D models built from the mesh of images and point cloud data captured using webcam data within Python.

The approach to this analysis will include collecting an image via a webcam. The image will be collected of various buildings and objects found around the Georgia Tech campus. From there, analysis will be performed on the geometry present and figure out the depth of the object using mesh/point cloud data. Point cloud data will allow detection of the orientation of the object relative to the camera. Generating a mesh will also allow simple transformations of the geometry to take place.

II. Methodology

The live image was captured using the CV2 toolbox in Python. The corresponding code can be seen below in Figure 1. CV2 allows the computer webcam to turn on. The user hitting the ‘s’ key captured the image and then the last part of this script saved it to a ‘.jpg’ file.

```
cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    cv2.imshow("Frame", frame)

    if cv2.waitKey(1) == ord('s'):
        break

# Convert the image to JPG format and save it
cv2.imwrite('image.jpg', frame, [cv2.IMWRITE_JPEG_QUALITY, 90])
# Get the full file path of the saved image

cap.release()
```

Figure 1. Image Generation from Webcam using CV2 toolbox.

The result of the code in Figure 1 would output and save an image like the following taken of the team members shown in Figure 2. The images that are processed in CV2 do not have to been directly taken from the webcam to be processed for the depth map. External images can be imported into the system as well, which can then be fed into the following algorithm for depth perception.



Figure 2. Computer output of image captured using CV2.

To create the Depth Perception Model, the GLPNImageProcessor and GLPNForDepthEstimation tools from the Transformer package were needed [1]. These two tools are from the Transformers package, and they rely on Machine Learning encoders to take in an image, which is then transformed into a depth map through the depth perception algorithm during the decoding process. To output the depth map by preprocessing the input image for monocular depth perception, the GLPNImageProcessor Tool was used as shown in Figure 3.

```
feature_extractor = GLPNImageProcessor.from_pretrained("vinvino02/glpn-nyu")
model = GLPNForDepthEstimation.from_pretrained("vinvino02/glpn-nyu")

# load and resize the input image
# image = Image.open('C:\\Users\\uniqu\\PycharmProjects\\pythonProject\\ME 6104\\FinalProject\\Mohan.jpg')
image = Image.open(file)
new_height = 480 if image.height > 480 else image.height
new_height -= (new_height % 32)
new_width = int(new_height * image.width / image.height)
diff = new_width % 32
new_width = new_width - diff if diff < 16 else new_width + 32 - diff
new_size = (new_width, new_height)
image = image.resize(new_size)

# prepare image for the model
inputs = feature_extractor(images=image, return_tensors="pt")

# get the prediction from the model
with torch.no_grad():
    outputs = model(**inputs)
    predicted_depth = outputs.predicted_depth

# remove borders
pad = 16
```

```
# remove borders
pad = 16
output = predicted_depth.squeeze().cpu().numpy() * 1000.0
output = output[pad:-pad, pad:-pad]
image = image.crop((pad, pad, image.width - pad, image.height - pad))

# visualize the prediction
fig, ax = plt.subplots(1, 2)
ax[0].imshow(image)
ax[0].tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)
ax[1].imshow(output, cmap='plasma')
ax[1].tick_params(left=False, bottom=False, labelleft=False, labelbottom=False)
plt.tight_layout()
plt.show()

FX_DEPTH = 5.8262448167737955e+02
FY_DEPTH = 5.8269103270988637e+02
CX_DEPTH = 3.1304475870804731e+02
CY_DEPTH = 2.3844389626620386e+02

pcd = []
height, width = output.shape
for i in range(height):
    for j in range(width):
        z = -output[i][j]
        x = -(j - CX_DEPTH) * z / FX_DEPTH
        y = -(i - CY_DEPTH) * z / FY_DEPTH
        pcd.append([x, y, z])
```

Figure 3. Code used for the generation of the depth perception model.

The result of the code shown above can be seen in the following Figure 4, where the cooler blue color indicates an object being closer to the camera and the hotter orange and pink/red colors indicate an object being much further away.

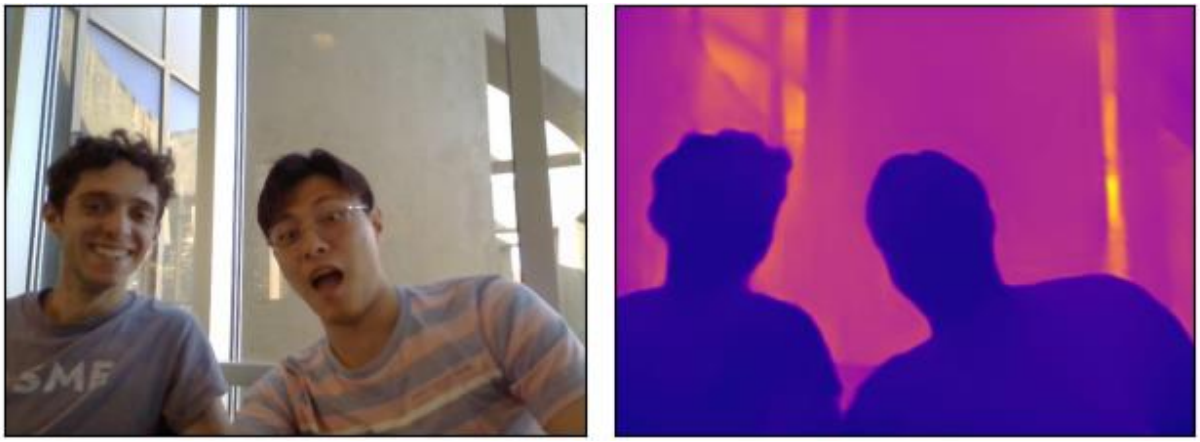


Figure 4. The resulting depth perception model shown next to the user image captured.

```
FX_DEPTH = 5.8262448167737955e+02
FY_DEPTH = 5.8269103270988637e+02
CX_DEPTH = 3.1304475870804731e+02
CY_DEPTH = 2.3844389626620386e+02

pcd = []
height, width = output.shape
for i in range(height):
    for j in range(width):
        z = -output[i][j]
        x = -(j - CX_DEPTH) * z / FX_DEPTH
        y = -(i - CY_DEPTH) * z / FY_DEPTH
        pcd.append([x, y, z])
```

Figure 5. Code used for generating PCD.

Point cloud data (PCD) is then generated using the methodology shown above in Figure 5. The focal and optical centers are determined through the recommendation provided from external sources. The Open3D Python package is then further used for 3D visualization [2]. A rotation algorithm is also directly applied to the generated PCD so that a user can rotate any image by a designated angle as shown in the code of Figure 6.

```
FX_DEPTH = 5.8262448167737955e+02
FY_DEPTH = 5.8269103270988637e+02
CX_DEPTH = 3.1304475870804731e+02
CY_DEPTH = 2.3844389626620386e+02

pcd = []
height, width = output.shape
for i in range(height):
    for j in range(width):
        z = -output[i][j]
        x = -(j - CX_DEPTH) * z / FX_DEPTH
        y = -(i - CY_DEPTH) * z / FY_DEPTH
        pcd.append([x, y, z])
```

```
def rot_pcd(pcd, angle_x, angle_y, angle_z):
    theta_x = np.radians(angle_x)
    theta_y = np.radians(angle_y)
    theta_z = np.radians(angle_z)
    Rz = np.array([[math.cos(theta_z), -math.sin(theta_z), 0], [math.sin(theta_z), math.cos(theta_z), 0], [0, 0, 1]])
    Ry = np.array([[math.cos(theta_y), 0, math.sin(theta_y)], [0, 1, 0], [-math.sin(theta_y), 0, math.cos(theta_y)]])
    Rx = np.array([[1,0,0],[0, math.cos(theta_x),-math.sin(theta_x)],[0, math.sin(theta_x),math.cos(theta_x)]])
    R = Rz*Ry*Rx

    pcd_arr_base = []
    height, width = output.shape
    for i in range(height):
        for j in range(width):
            z = -output[i][j]
            x = (j - CX_DEPTH) * z / FX_DEPTH
            y = (i - CY_DEPTH) * z / FY_DEPTH

            pcd_arr_base.append(R@[x, y, z])

    return pcd_arr_base
```

Figure 6. Code used for the generation of the PCD from the Depth Perception Model.

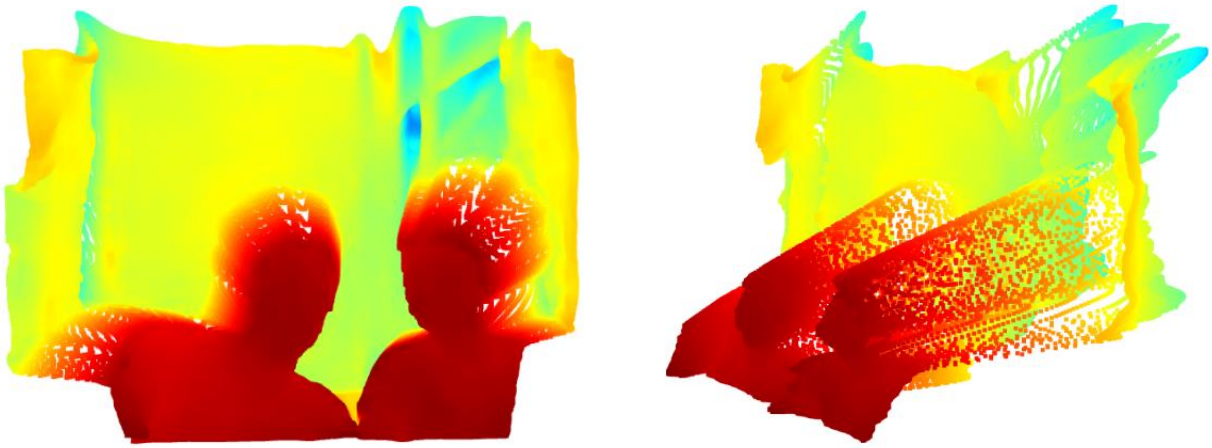


Figure 7. Generated Point Cloud Data shown on the left. Rotated PCD is shown on the right.

The results are displayed in Figure 7, where the point cloud data is generated from the depth map shown in Figure 4 using the O3D visualization package in Python, as shown in the code of Figure 8 down below. As shown, the red points represent the points closer to the viewer while the points in brighter (yellow, green, and blue) represent those that are more distant. In Figure 7, the figure on the right represents the PCD that is slightly rotated to show the relative locations of the PCD with respect to each other.

```
def plot(pcd,title):

    pcd_o3d = o3d.geometry.PointCloud()  # create point cloud object
    pcd_o3d.points = o3d.utility.Vector3dVector(pcd)  # set pcd_np as the point cloud points
    # Visualize:
    o3d.visualization.draw_geometries([pcd_o3d], window_name=title)



pcd_1 = rot_pcd(pcd,0,0,0)
plot(pcd_1,'No Rotation')
```

Figure 8. Plotting of PCD using O3D.

The series of steps done in the methodology thus far were then repeated for numerous other images and the results can be seen in the section that follows.

III. Results

The methodology was then followed for numerous other images taken around the Georgia Tech Campus. Figure 8 shows an example of a chair that is fed into the algorithm to generate the depth map as well as the PCD's in different orientations shown in Figure 9.

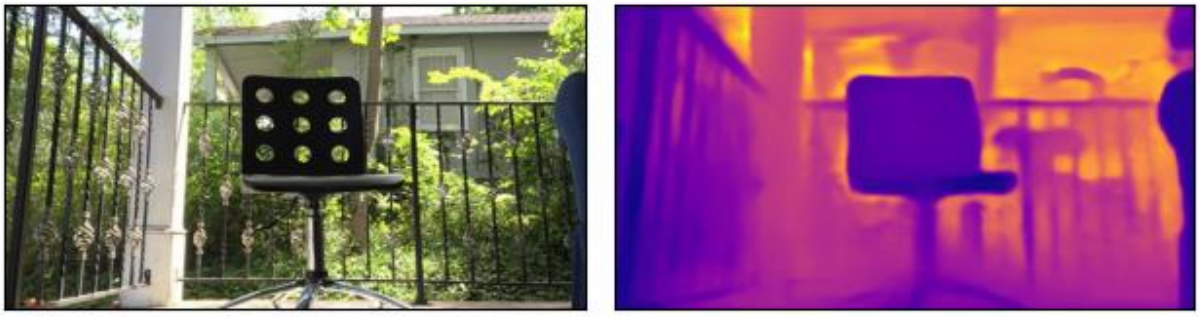


Figure 7. Chair Image and corresponding Depth Perception Model.

Figure 9 below shows how the depth perception model is sensitive to the sunlight and isn't really all that great at illustrating the empty space in between the bars on the side of the porch as well as in the holes designed within the chair. These discrepancies are further propagated visually by the wild looking red lines that protrude outwards as seen in the top right and bottom left of Figure 9.

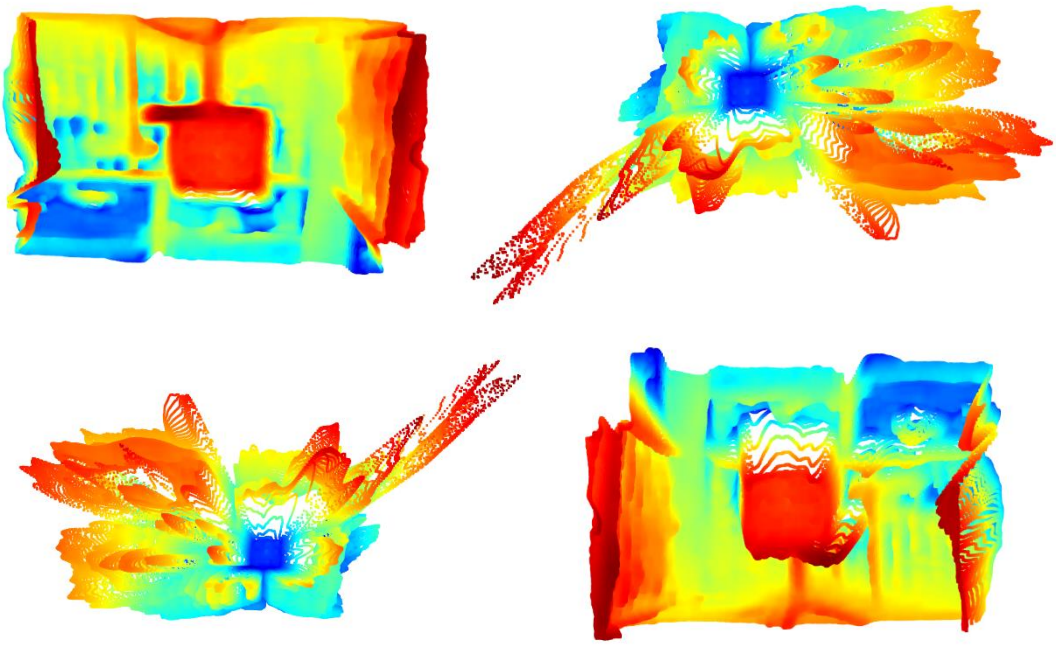


Figure 8. The Point Cloud Data of the Chair is shown at the bottom right. Different Orientations of the PCD are displayed as 180-degree Rotation in X-axis shown at the top left, 180-degree Rotation in Y-Axis shown at the Bottom left, and the 180-degree Rotation in Z-axis shown at the top right.

Another example is done with the Tech Tower shown in Figure 10. The same procedure is done for this example.

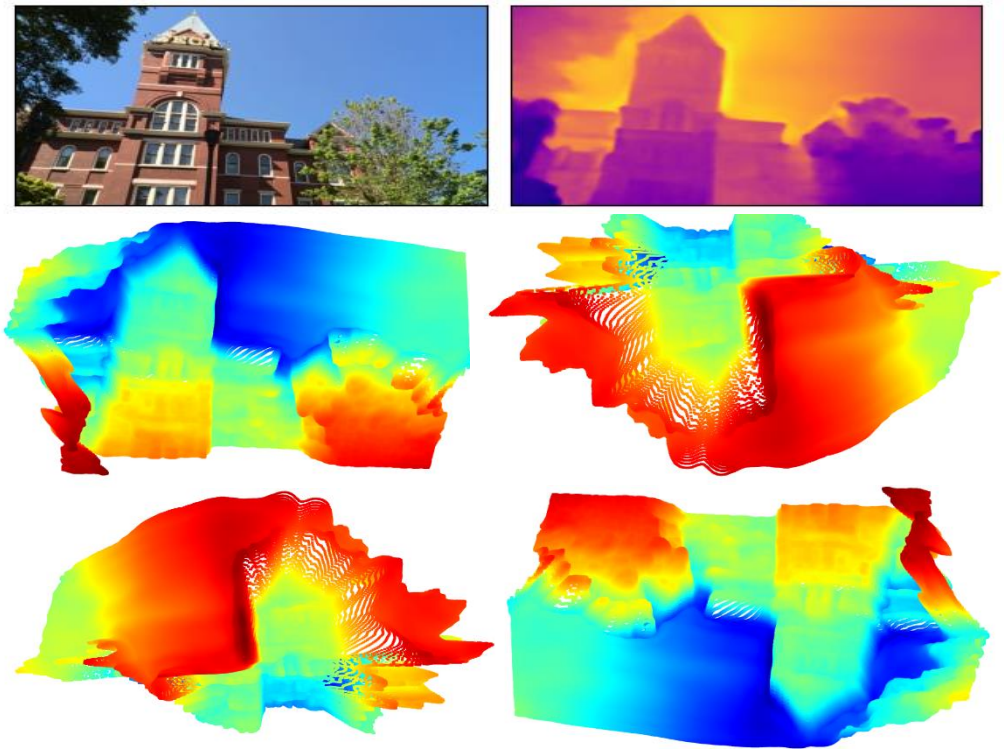


Figure 9. The Tech Tower image and the depth map are shown at the top. The Point Cloud Data is shown at the middle left. Different Orientations of the PCD are displayed as 180-degree Rotation in X-axis shown at the bottom right, 180-degree Rotation in Y-Axis shown at the Bottom left, and the 180-degree Rotation in Z-axis shown at the middle right.

Some additional examples include Einstein shown in Figure 11 shown below. The same procedure is done for this example.

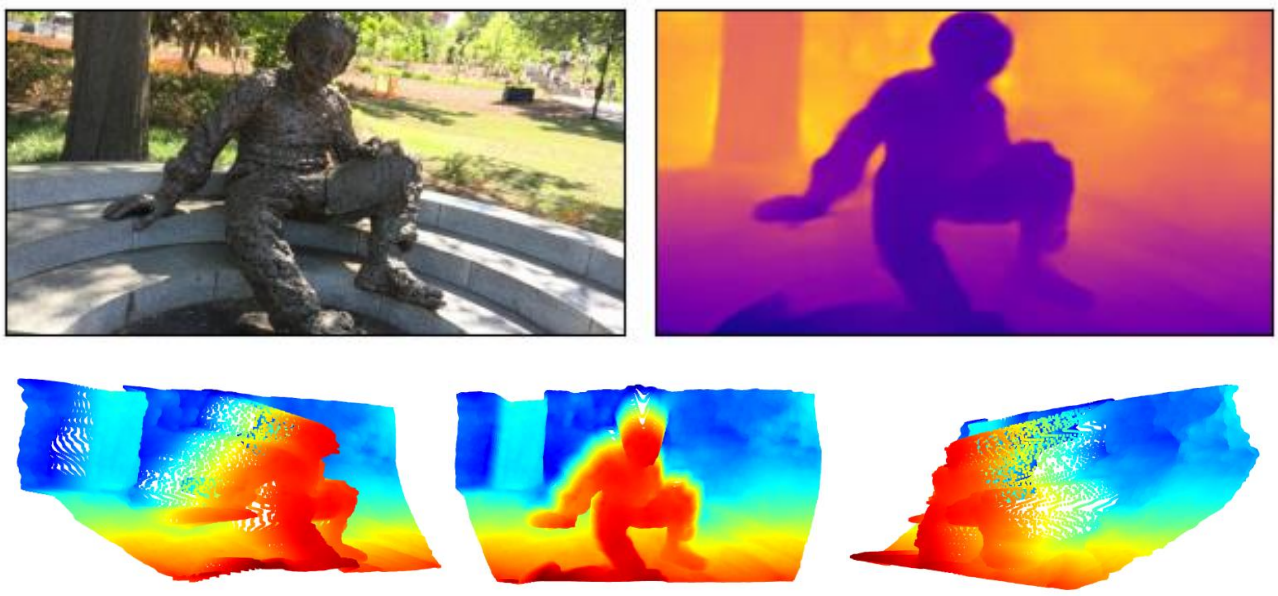


Figure 11. The Einstein image and the depth map are shown at the top. The Point Cloud Data of the Einstein are shown below, where the bottom left and right displays the rotated PCD.

IV. Summary and Future Work

Some of the future works include optimizing the depth perception algorithm that is currently being used for this project. Since the depth map performance is impacted by the existence of sunlight, the team members can investigate ways to reduce the influence of it on the depth perception. Objects being too close or too far away, or even simply the same distance away, have all caused issues in the depth perception recognition method used. Further refinement is necessary to distinguish between different objects at varying distances from the camera. Also, aspects like image resolution can be improved through Lidar scanning (as seen in robotics) and run times can be reduced through simpler and more efficient code. Additional studies should be conducted to find ways for image detection to work better with distant objects.

Some further future works include calibrating the focal origin and optical centers of the camera to avoid skewed depth perception since the current rotational transformation algorithm is incorrect since it does not account for the focal origin location. Currently, the focal origin is set to that which was defined in Figure 5 as recommended by online literature [2]. This can be improved through conceptual changes such as translating the Point Cloud Data to the computer defined origin, applying the transformation (rotation) from the new position, and then translating the PCD back based on the initial translation distance.

It would also be a huge advancement in this analysis to be able to generate a mesh using the Ball-Pivoting method or the Poisson Reconstruction methods [3] since this would allow for a part to be captured, analyzed, and then generated in 3D for a CAD modelling software in under a minute.

V. Reference

[1] Gatti, M. (2023, March 6). Generate a 3D mesh from an image with python. Medium. Retrieved April 24, 2023, from https://towardsdatascience.com/generate-a-3d-mesh-from-an-image-with-python-12210c73e5cc

[2] Zatout, C. (2023, January 25). Estimate point clouds from depth images in Python. Medium. Retrieved April 24, 2023, from https://betterprogramming.pub/point-cloud-computing-from-rgb-d-images-918414d57e80

[3] Florent Poux, P. D. (2021, April 11). *5-step guide to generate 3D meshes from point clouds with python*. Medium. Retrieved April 25, 2023, from https://towardsdatascience.com/5-step-guide-to-generate-3d-meshes-from-point-clouds-with-python-36bad397d8ba